Hardware/Software Design Description

For

CourtSweeper: A Vision-Based Tennis Ball Retriever with Dual-Roller Intake Mechanism

Version 1.0 approved

Group 13

Date 2026-03-23

Revision History

Name	Date	Reason For Changes	Version
Yilin Zhang	2026-03-17	Initial document creation and LaTeX template configuration.	0.1
Yingrui Sun	2026-03-18	Drafted initial hardware specifications, including dual-roller mechanism and power system.	0.2
Shijia Luo	2026-03-18	Drafted software architecture, including machine vision algorithms and communication protocols.	0.3
Yuwei Zhao	2026-03-23	Comprehensive review and optimization. Finalized system design and approved Baseline 1.0.	1.0

Table of Contents

1 Introduction **1**
1.1 Purpose 1
1.2 Scope 1
1.3 Definitions, Acronyms and Abbreviations 2
1.4 References 3
1.5 Organization of Document 3

2 HARDWARE DESIGN **4**
2.1 Camera Setup 4
2.2 DJI RoboMaster EP Chassis 5
2.3 Custom Intake Mechanism 5
2.4 LiDAR Sensor Setup 6
2.5 Jetson Orin NX Setup 7
2.6 Power Management System 8

3 SOFTWARE MODULES DESIGN **9**
3.1 Architecture Overview 9
3.1.1 Chassis Navigation & DJI SDK Integration 9
3.1.2 SLAM & LiDAR Localization Package 10
3.1.3 Workspace Gridding & Calibration Package 10
3.1.4 Coverage Path Planning Package 10
3.1.5 Video I/O & Pre-processing Package 10
3.1.6 Video Processing Package (YOLO & OpenCV) 10
3.1.7 FSM-Based Event Recognition & Control Package 11
3.2 Class Design 11

4 USER INTERFACE DESIGN **13**
4.1 Main Dashboard & Video HUD 13
4.2 Teleoperation & Control Panel 13
4.3 Telemetry & Log Console 14

1 INTRODUCTION

1.1 Purpose

This document describes the preliminary design of the CourtSweeper system. Its primary purpose is to provide a comprehensive architectural overview and detailed engineering specifications, serving as the Baseline 1.0 blueprint for the development team. It details the physical mechanical structures, electronic power systems, computer vision algorithms, and the iOS-based user interface to guide the implementation, integration, and testing phases.

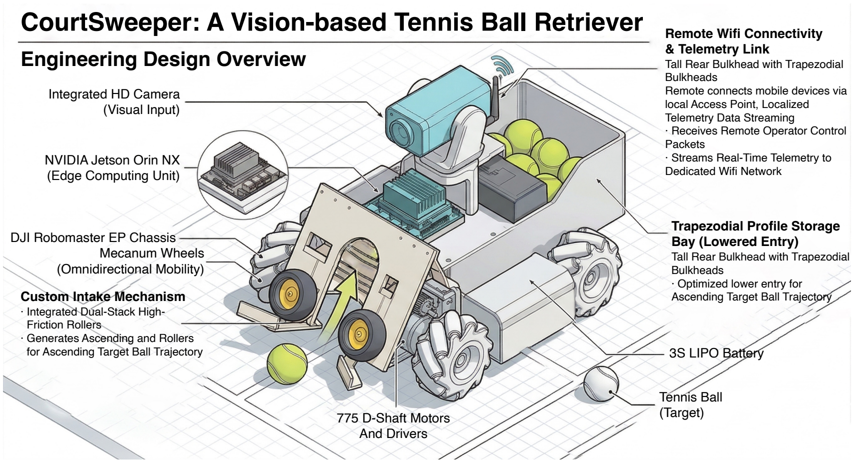

Figure 1: Comprehensive engineering design overview of the CourtSweeper robotic platform.

1.2 Scope

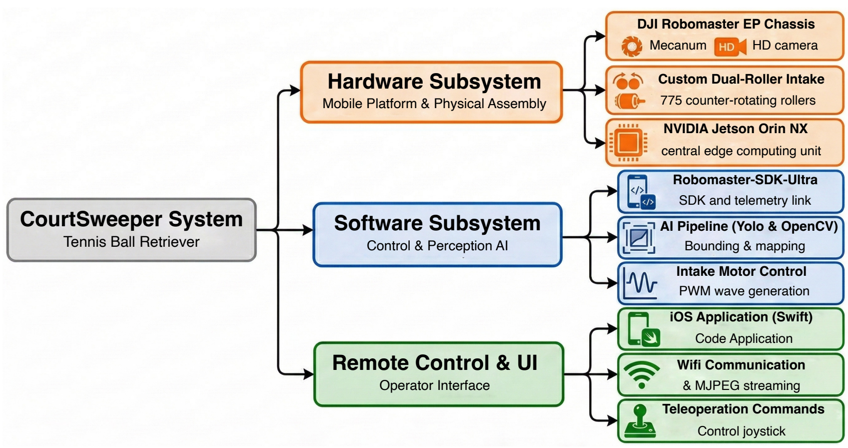

Figure 2: Hierarchical scope tree diagram detailing the overarching architecture of the CourtSweeper system.

This document describes the design of the CourtSweeper, an intelligent, vision-guided robotic vehicle designed to automate the retrieval of scattered tennis balls in sports facilities. The scope of this system encompasses:

- **Hardware Subsystem:** A hybrid mobile platform utilizing the DJI RoboMaster EP [2] chassis to provide omnidirectional mobility and the primary HD camera feed. It is physically integrated with a custom-built, high-torque dual-roller intake mechanism, which is driven by 775 D-shaft motors and BTS7960 drivers for ball collection, and an NVIDIA Jetson Orin NX [4] acting as the central edge computing unit.
- **Software Subsystem:** An onboard AI pipeline utilizing the RoboMaster-SDK-Ultra [5] (a customized, high-performance fork of the official DJI SDK developed specifically for this project) for vehicle telemetry and camera stream extraction. This is combined with the YOLO [6] object detection model and OpenCV, enabling real-time target recognition, spatial coordinate mapping, and dynamic intake motor control via PWM.
- **Remote Control & UI:** A native iOS application [1] developed in Swift, establishing a localized Wi-Fi connection to stream AI-annotated video feeds (via MJPEG) and transmit low-latency teleoperation commands to the Jetson unit.

1.3 Definitions, Acronyms and Abbreviations

Table 1: List of abbreviations and terminology used in the CourtSweeper.

Term	Definition
HSDD	Hardware/Software Design Description.
CS	CourtSweeper, the designated project name for the tennis ball collecting robot.
UI	User Interface, specifically referring to the iOS application in this project.
YOLO	You Only Look Once (specifically yolo26), a SOTA real-time object detection system utilized for high-speed tennis ball recognition.
RoboMaster-SDK-Ultra	A customized, advanced fork of the official DJI SDK (developed by RamessesN), utilized to interface with the RoboMaster EP chassis, execute complex telemetry, and extract raw HD camera feeds.
OpenCV	Open Source Computer Vision Library, utilized for image processing, color space conversion, and bounding box coordinate extraction.
FSM	Finite State Machine, the logic architecture governing the autonomous and manual behavioral states of the robot.
IPC	Inter-Process Communication, a data exchange mechanism between OS processes. The custom SDK circumvents IPC to minimize video latency.
Mecanum Wheels	Omnidirectional wheels on the RoboMaster EP chassis that allow the robot to move in any direction without changing its orientation.
PWM	Pulse Width Modulation, utilized for custom motor speed regulation in the 775 motor-driven intake mechanism.
LiPo	Lithium Polymer, the high-discharge-rate battery chemistry utilized for the heavy-duty 3S actuation power source.
MJPEG	Motion JPEG, a video compression format used to stream AI-processed (bounding-box annotated) video frames to the iOS client with low latency.
HUD	Heads-Up Display, the primary transparent-style layout utilized in the iOS application for concurrent video and telemetry monitoring.
AP Mode	Access Point Mode, allowing the Jetson Orin NX to act as a localized Wi-Fi router for direct mobile connection in outdoor environments.

1.4 References

[1] Apple Inc. *Apple Developer Documentation - Network Framework*. 2026. URL: `https://developer.apple.com/documentation`.

[2] DJI. *RoboMaster EP Programming Guide and SDK Documentation*. 2020. URL: `https://www.dji.com/cn/robomaster-ep`.

[3] *Dual Roller Mechanism*. 2024. URL: `https://www.bilibili.com/video/BV1bgveeoEiX?vd_source=6621406a814999260edb3d6efe8942da`.

[4] Nvidia Inc. *NVIDIA Jetson Orin NX*. 2022. URL: `https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/`.

[5] RamessesN. *Robomaster SDK Ultra*. 2025. URL: `https://github.com/RamessesN/Robomaster-SDK-Ultra`.

[6] *Ultralytics YOLO26*. 2026. URL: `https://docs.ultralytics.com/models/yolo26/`.

1.5 Organization of Document

To provide a structured and logical flow of the system architecture, this document is organized into the following major sections:

Table 2: Organization of this Hardware/Software Design Description.

Section	Core Content	Description
Section 2 *Hardware Design*	• DJI RoboMaster EP Chassis • Custom Intake Mechanism • Jetson Orin NX Setup • Power Management System	Provides the physical and electrical blueprint of the robot. It details the integration of the hybrid mobile platform, the custom 775 motor-driven dual-roller mechanism, and the multi-voltage power distribution topology.
Section 3 *Software Design*	• DJI SDK Integration • Video Pre-processing • YOLO and OpenCV Pipeline • FSM-Based Control Logic • Class Design & Architecture	Describes the overarching onboard AI architecture. It breaks down the integration of the custom SDK for chassis navigation, the computer vision pipeline for target detection, and the finite state machine governing the intake motors.
Section 4 *UI Design*	• Main Dashboard & Video HUD • Teleoperation & Control Panel • Telemetry & Log Console	Outlines the Swift-based iOS application architecture, detailing the visual presentation layer, network socket communication, and real-time system monitoring protocols.

2 Hardware design

The hardware architecture of the CourtSweeper system is a hybrid design, integrating a commercial robotic platform with custom-engineered mechanical and computational subsystems. This section details the physical embodiment, sensor configurations, and electrical topologies required to achieve autonomous tennis ball retrieval.

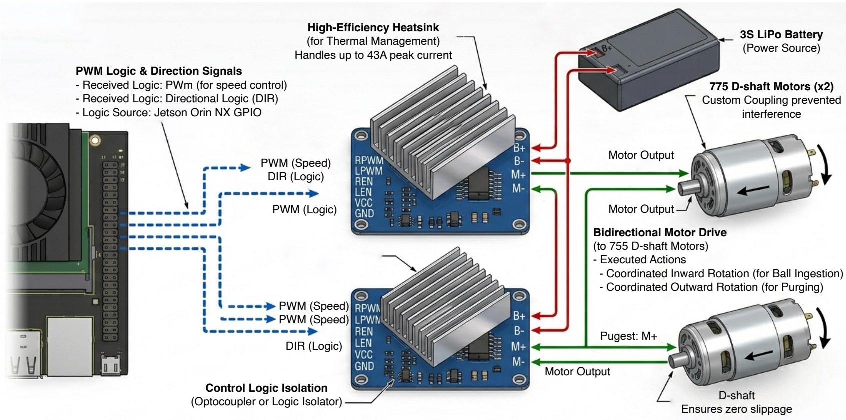

Figure 3: An engineering diagram detailing the BTS7960 high-current drive architecture for the CourtSweeper system, illustrating the connection of an NVIDIA Jetson Orin NX to two BTS7960 motor drivers powered by a 3S LiPo battery to control two 775 D-shaft motors, featuring logic signal connections, power distribution, bidirectional motor drive, and technical callouts for key components like heatsinks and motor features.

2.1 Camera Setup

The primary visual input via the integrated HD camera on the Robomaster EP chassis, which feeds raw video streams to the edge computing unit for target detection. The core hardware parameters of the integrated camera are detailed in Table 3.

Figure 4: A physical picture of Robomaster EP Camera Module.

Table 3: DJI RoboMaster EP Integrated Camera Specifications

Hardware Parameter	**Specification**
Sensor Type	CMOS 1/4″, Effective Pixels: 5 MP
Field of View (FOV)	120°
Max Video Resolution	FHD: 1080p@30fps / HD: 720p@30fps
Max Video Bitrate	16 Mbps
Max Still Photo Resolution	2560 × 1440

2.2 DJI RoboMaster EP Chassis

Figure 5: A physical picture of Robomaster EP Chassis Module.

The foundation of the CourtSweeper is the DJI RoboMaster EP chassis. Selected for its industrial-grade stability and payload capacity, the chassis is equipped with four Mecanum wheels, granting the robot true omnidirectional mobility. This holonomic drive system is crucial for the precise spatial adjustments required to align the front-facing intake mechanism with scattered tennis balls without needing a turning radius.

2.3 Custom Intake Mechanism

To achieve reliable and rapid ball collection, a custom dual-roller intake mechanism [3] was designed and mounted at the front of the chassis. The actuation is driven by two high-torque 775 DC motors (D-shaft configuration, rated at 12V and 4600 RPM).

- **Mechanical Coupling:** The motors are mated to 60mm solid rubber wheels featuring high-friction hexagonal grooves via 30mm extended brass couplings. This extended design provides essential clearance to prevent mechanical interference between the rotating wheel hubs and the motor casing, while the D-shaft ensures zero slippage under high torque.
- **Motor Drivers:** Two BTS7960 high-current motor driver modules (capable of handling up to 43A peak current) are utilized to independently control the two motors. They receive Pulse Width Modulation (PWM) and directional logic signals from the central computing

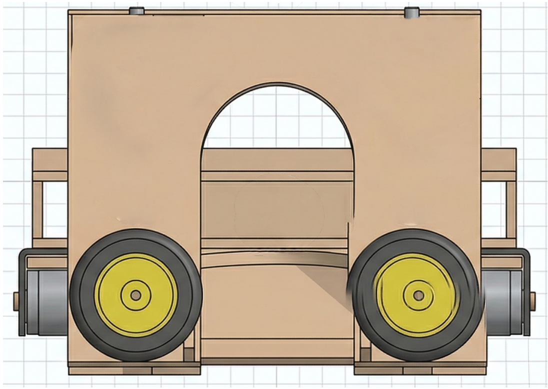

Figure 6: A detailed front-on view of the Courtsweeper dual-roller intake mechanism.

unit to execute coordinated inward rotation (for ball ingestion) or outward rotation (for purging).

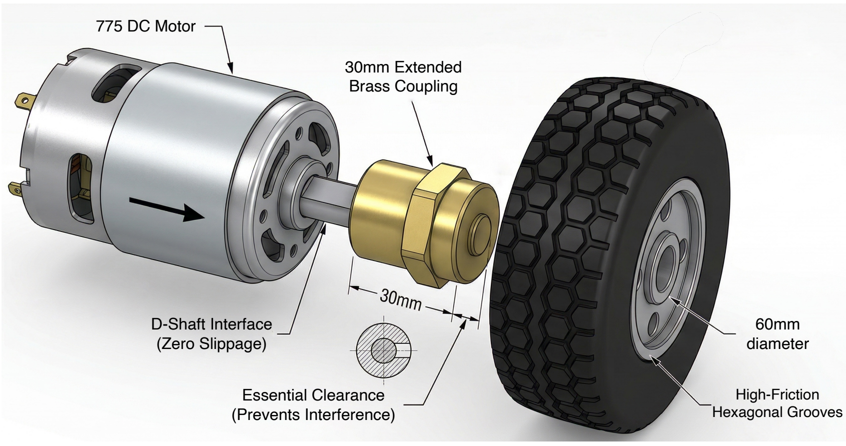

Figure 7: Close-up view of the mechanical coupling. The 30mm extended brass coupling provides critical operational clearance between the 775 motor casing and the high-friction intake roller.

2.4 LiDAR Sensor Setup

To ensure precise operational boundary gridding and efficient path planning (as outlined in the software architecture Fig.11), the Courtsweeper incorporates a dedicated 2D LiDAR sensor. Based on a triangulation principle, this sensor provides robust 2D scanning of the court environment and potential obstacles.

- **Sensor Details:** The selected hardware is the YDLIDAR X3-YB-1 triangulation-based LiDAR. It is mounted at the rear of the chassis and interfaces with the Jetson Orin NX via a customized USB-serial connection.

- **Specifications:** The X3-YB-1 offers a maximum distance range of up to 10 meters, enabling comprehensive court coverage. Key operational parameters include a sampling frequency of 4000 Hz, a standard rotating frequency of 7 Hz, and an angular resolution of 0.6°.

Figure 8: A physical picture of the YDLIDAR X3-YB-1 LiDAR sensor.

2.5 Jetson Orin NX Setup

The cognitive core of the system is the NVIDIA Jetson Orin NX. Acting as the Edge Computing Unit, it is responsible for handling computationally intensive tasks locally, thereby eliminating cloud-dependency and minimizing latency. The Jetson module runs the Ubuntu operating system and is configured to:

1. Execute the YOLO object detection pipeline concurrently with OpenCV operations to process the H.264 video stream captured by the DJI chassis.
2. Run the customized RoboMaster-SDK-Ultra to translate bounding box coordinates into proportional navigation commands.
3. Generate precise PWM signals via its GPIO pins to command the BTS7960 drivers.
4. Operate in Access Point (AP) mode to establish a localized Wi-Fi network for the iOS UI client.

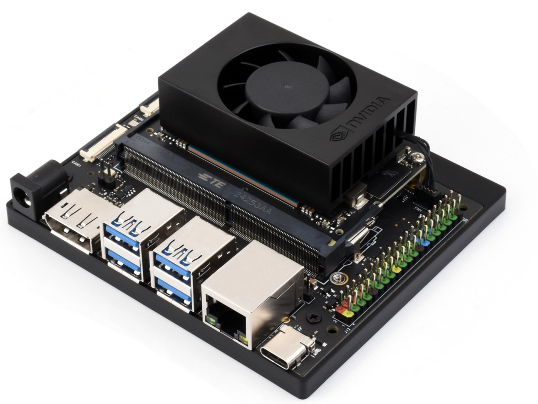

Figure 9: A physical picture of NVIDIA jetson Orin NX.

2.6 Power Management System

Given the diverse voltage requirements and the risk of electromagnetic interference from high-current motors, the CourtSweeper employs an isolated, multi-rail power distribution topology:

- **Mobility Power:** The DJI RoboMaster EP chassis and its onboard camera are powered autonomously by the proprietary DJI Intelligent Battery (3S LiPo, Nominal 10.8V, 2400mAh, 25.92Wh).
- **Actuation Power:** The 775 intake motors and BTS7960 drivers are powered by a dedicated 3S (11.1V) Lithium Polymer (LiPo) battery rated at 5200mAh and 40C discharge. This ensures the motors receive sufficient burst current during ball compression without inducing voltage sags in the logic circuits. A low-voltage buzzer (BX100) is attached to the balance lead for real-time cell monitoring.

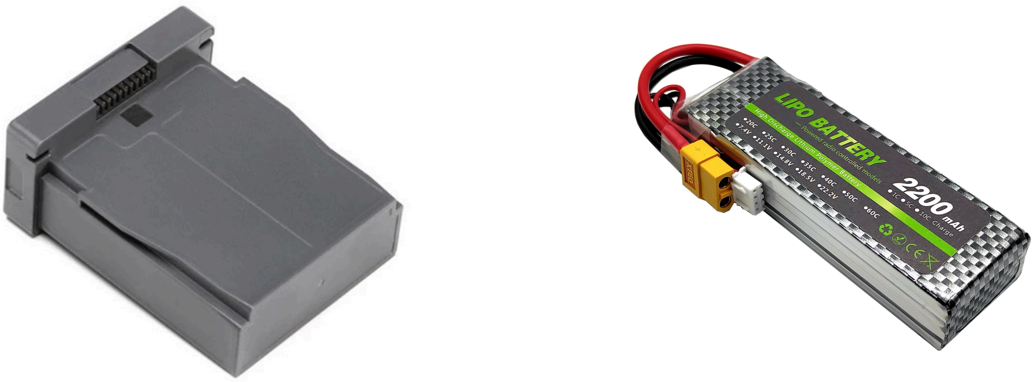

Figure 10: A physical picture of Robomaster EP battery & Motor-driven 3S LiPo battery.

- **Logic Power:** The Jetson Orin NX requires a stable 19V input. To prevent logic resets caused by motor back-EMF, it is powered by a galvanically isolated power source.
- **Common Grounding:** Crucially, the ground (GND) planes of the Jetson Orin NX and the BTS7960 motor drivers are tied together to ensure reliable logic-level signal transmission.

3 SOFTWARE MODULES DESIGN

3.1 Architecture Overview

The software subsystem of CourtSweeper operates primarily on the NVIDIA Jetson Orin NX edge computing unit, utilizing a multi-threaded pipeline designed for high-throughput and low-latency operation. The architecture integrates heterogenous sensory inputs—comprising the high-definition DJI camera stream and 2D LiDAR range data—to establish full situational awareness.

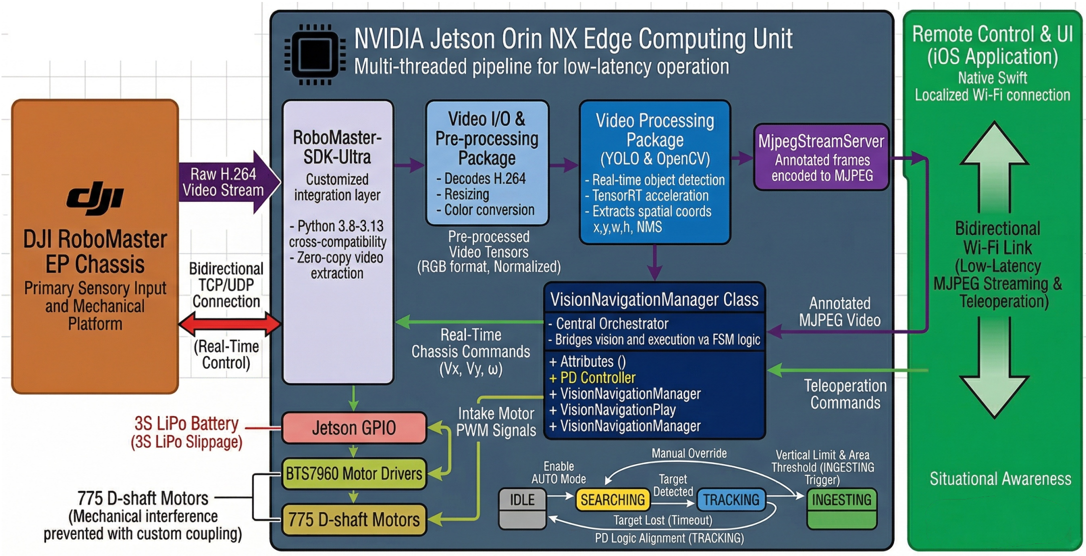

Figure 11: Detailed comprehensive diagram of the CourtSweeper software subsystem architecture, integrating visual perception and SLAM-based navigation pipelines.

The core logic bridges these sensory inputs with the mechanical outputs (Mecanum chassis navigation and 775 motor actuation) through an advanced decision-making engine. This engine combines real-time SLAM (Simultaneous Localization and Mapping), heuristic workspace gridding, complete coverage path planning, and deep learning-based object detection, all governed by a robust Finite State Machine (FSM).

3.1.1 Chassis Navigation & DJI SDK Integration

To interface with the mobile platform, the system implements the **RoboMaster-SDK-Ultra**, a customized integration layer offering native cross-compatibility across Python 3.8 $\sim$ 3.13. This allows DJI hardware drivers to coexist in the same execution environment as modern YOLO, OpenCV, and ROS-based SLAM pipelines, circumvents severe Inter-Process Communication (IPC) overhead when transferring large data matrices.

Operationally, this unified SDK manages a high-speed connection over the local network. It directly extracts the raw H.264 video stream from the integrated camera for zero-copy visual processing, while concurrently transmitting translational (V_x, V_y) and rotational (ω) velocity commands back to the holonomic drive system based on calculated global paths or local reactive maneuvers.

3.1.2 SLAM & LiDAR Localization Package

To provide the robot with precise spatial awareness, this package interfaces with the LiDAR via USB-serial. It implements a robust SLAM algorithm accelerated by the Jetson's GPU to process scan matching in real-time. This module is responsible for:

- **Map Generation:** Constructing a 2D occupancy grid map of the surrounding environment during the initial setup phase.
- **Real-time Localization:** Providing continuous, high-frequency estimates of the robot's pose (x, y, θ) within the generated map by fusing LiDAR odometry and chassis wheel odometry data.

3.1.3 Workspace Gridding & Calibration Package

This module is utilized during system initialization to define the operational boundaries.

- **Court Calibration:** The operator manually drives the robot along the perimeter of the tennis court playing area. The package records these pose boundaries within the SLAM map to isolate the sweeping workspace.
- **Grid Discretization:** The calibrated workspace is discretized into a uniform grid map. Each grid cell dimensions correspond to the sweeping width of the intake mechanism. Cells are classified as occupied (by static obstacles), traversable, cleared, or containing a target (projected from visual data).

3.1.4 Coverage Path Planning Package

This package is the core efficiency optimizer. In the absence of visually detected tennis balls, it ensures the entire traversable court area is swept systematically.

- **Global Planning:** Utilizing algorithms such as A* or Spanning Tree Coverage, it generates an optimal coverage path through all traversable grid cells to minimize sweeping time and redundant traversal.
- **Local Planning & Replenishment:** If visual processing detects a ball, the global coverage path is paused. After ball ingestion, the planner calculates a local trajectory back to the last paused waypoint on the global coverage path to resume systematic sweeping.

3.1.5 Video I/O & Pre-processing Package

Before AI inference, the raw H.264 video feed is decoded into discrete frame matrices. The pre-processing pipeline scales the HD frames to specified YOLO input dimensions and converts color spaces to RGB, normalizing pixel values to a $[0, 1]$ range for optimized neural network ingestion.

3.1.6 Video Processing Package (YOLO & OpenCV)

Utilizing the YOLO architecture accelerated by TensorRT, this module performs real-time object detection to identify tennis balls. OpenCV functions extract the spatial coordinates of the bounding box $(x, y, width, height)$ and calculate the target centroid.

Crucially, by utilizing the current robot pose from the SLAM package and known camera intrinsic parameters, this module projects the image-plane coordinates onto the ground plane to estimate the ball's (x, y) location in the global map, updating the target status of specific grid cells.

3.1.7 FSM-Based Event Recognition & Control Package

The robot's behavioral logic is strictly governed by a Finite State Machine (FSM), decoupling perception from mechanical actuation. The optimized state transitions are as follows:

- **State 0 - IDLE:** Awaiting commands. Manual teleoperation via iOS UI overrides all behaviors. Court calibration is performed in this state.
- **State 1 - SEARCHING (Optimized Coverage):** If no ball is visuals detected, the FSM enters searching state. Instead of random patrol, the chassis follows the pre-generated **Complete Coverage Path**. The system continuously ticks off grid cells as "cleared" upon traversal.
- **State 2 - TRACKING (Target Alignment):** Upon visual detection, FSM transitions to Tracking. Global path following is paused. A PD controller uses the camera centroid offset for rapid alignment, while a local planner drives the chassis toward the projected map coordinates of the ball.
- **State 3 - INGESTING (Intake Trigger):** As defined previously, triggered by bounding box area thresholds indicating physical contact. Motors activate via PWM. Upon successful ingestion, the corresponding grid cell is marked as "cleared," and the FSM transitions back to SEARCHING, utilizing the CPP package to return to the paused coverage path.

3.2 Class Design

CourtNavigationManager Class Description

To support the integrated SLAM and Planning pipelines, the central orchestrator is upgraded to the `CourtNavigationManager`. It bridges visual perception, spatial mapping, gridding, and path planning modules.

Attribute	Description
Class Name	`CourtNavigationManager`
Aggregated Classes	`YoloDetector`, `SlamLocalizer`, `GridMapHandler`, `CoveragePathPlanner`, `DjiChassisController`, `MotorPwmDriver`
Associated Classes	`MjpegStreamServer`, `RosBridgeServer` (for LiDAR data)
Data Members	• `current_state`: Enum (IDLE, CALIBRATING, SEARCHING, TRACKING, INGESTING). • `robot_pose_global`: Pose (x, y, θ) provided by `SlamLocalizer`. • `discretized_grid`: 2D Array/Structure managing cell states (Occupied, Swept, Ball). • `global_coverage_path`: List of Waypoints to ensure 100% coverage.
Member Functions	• `update_master_state()`: High-level FSM logic switching based on vision, path progress, and grid status. • `calibrate_court_perimeter()`: Records boundary poses during manual driving to define grid limits. • `generate_gridded_workspace()`: Discretizes calibrated area and calls `CoveragePathPlanner` for the initial path. • `compute_navigation_command()`: In SEARCHING, computes commands to follow the coverage path. In TRACKING, switches to PD-based visual servoing and projected coordinate driving.
Processing	Operates in a dedicated high-priority thread. It fuses pose data and vision data to update the grid map, executes the FSM, dispatches velocity commands to the DJI SDK, and triggers intake motors.

4 USER INTERFACE DESIGN

This section describes the user interface design of the CourtSweeper iOS application. Developed natively in Swift using the SwiftUI framework, the mobile application serves as the primary remote command center. It connects directly to the Jetson Orin NX's localized Wi-Fi hotspot, providing real-time situational awareness, manual teleoperation capabilities, and system telemetry monitoring.

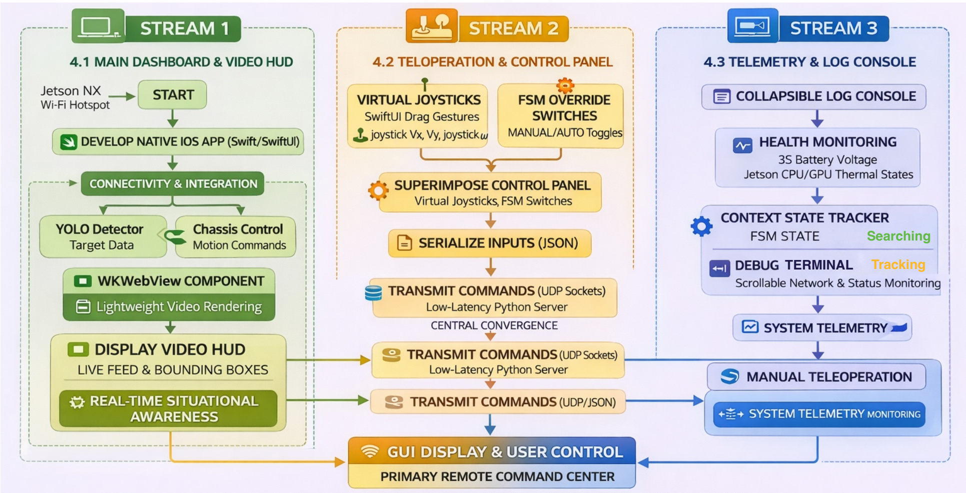

Figure 12: The software architecture and data flow for the CourtSweeper iOS application, illustrating the centralized SwiftUI core connected via bidirectional Wi-Fi and UDP sockets to external Jetson computing units, and featuring functional breakdowns of main dashboard, teleoperation, and telemetry modules.

4.1 Main Dashboard & Video HUD

The primary interface is designed as a landscape-oriented Heads-Up Display (HUD) to maximize the operator's field of view. The core component of this view is the video presentation layer.

- **Video Streaming Rendering:** Instead of relying on complex and latency-prone H.264 decoders on the client side, the app utilizes a lightweight `WKWebView` component to render the MJPEG video stream broadcasted by the Jetson unit.
- **AR Overlay:** The AI-processed bounding boxes, target crosshairs, and ball coordinates are overlaid directly onto the video feed, providing the operator with intuitive, real-time feedback on the YOLO perception module's performance.

4.2 Teleoperation & Control Panel

Superimposed on the lower sections of the HUD is the Control Panel, designed for ergonomic thumb access on an iPhone or iPad.

- **Virtual Joysticks:** Implemented via SwiftUI drag gestures, the left joystick dictates the omnidirectional translational movement (V_x, V_y) of the Mecanum chassis, while the right joystick controls rotational velocity (ω).

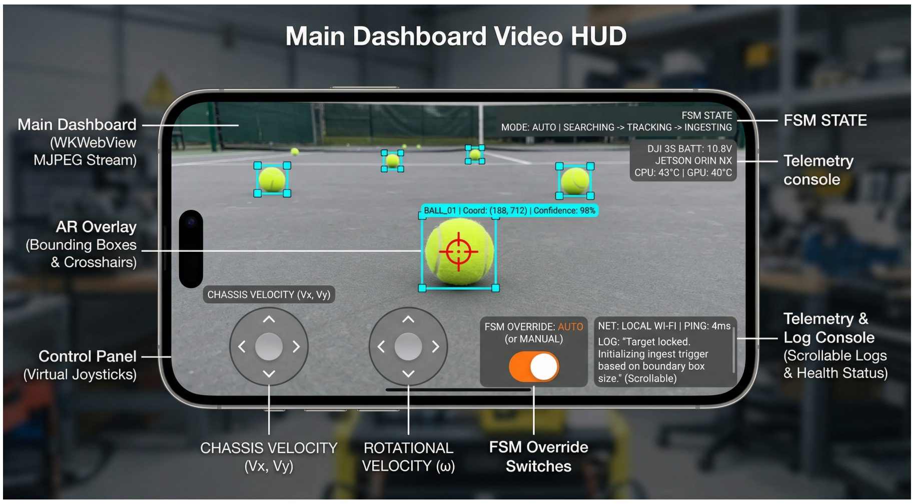

Figure 13: User interface layout of the CourtSweeper iOS application. The primary HUD renders the low-latency MJPEG video stream overlaid with real-time YOLO bounding boxes. The lower panel provides virtual joysticks for holonomic teleoperation and manual override switches for the system's Finite State Machine (FSM).

- **FSM Override Switches:** A prominent set of toggle buttons allows the user to switch the robot's FSM between `MANUAL` (teleoperation) and `AUTO` (autonomous searching and ingesting) states.
- **Command Transmission:** Control inputs are serialized into lightweight JSON packets and transmitted to the Jetson's Python server via low-latency UDP sockets using Apple's native `Network` framework.

4.3 Telemetry & Log Console

To facilitate debugging and monitor system health during field tests, a collapsible Log Console is integrated into the interface.

- **Health Monitoring:** Displays critical hardware telemetry parsed from the DJI SDK, including the 3S battery voltage (preventing over-discharge) and the Jetson's CPU/GPU thermal states.
- **State Machine Tracker:** A dynamic label continuously reflects the robot's current FSM state (e.g., *SEARCHING* $\rightarrow$ *TRACKING* $\rightarrow$ *INGESTING*).
- **Debug Terminal:** A scrolling text view (implemented via `ScrollView` in SwiftUI) outputs real-time network connection statuses, ping latency, and critical error logs thrown by the Python backend.